Ride-Hailing Order and Trip Dispatch Database

(Topic 5)

CDS 534 Database Management — Group Project

Group: **Royal Family**

A · Business & Requirements	CHEN Yinan (3162550)
B · ER Modelling	ZHANG Yijie (3162641)
C · Implementation	CHEN Yanwu (3162744)
D · Queries & Experiments	LING Tong (3162677)
E · PM & Integration	HU Zhongyuan (3162304)

November 8, 2026

Contents

1 Business Context and Goals **2**
- 1.1 Background and Business Process . . . 2
- 1.2 Stakeholders and Their Data Needs . . . 2
- 1.3 Quantified Goals, Constraints and Trade-offs . . . 2
- 1.4 Dataset Selection Rationale . . . 3

2 Entity-Relationship Model **4**
- 2.1 Notation and Conventions . . . 4
- 2.2 Official Five-Entity Base . . . 5
- 2.3 Approved Extensions . . . 5
- 2.4 Normalization . . . 6

3 Implementation **6**
- 3.1 Environment and Architecture . . . 7
- 3.2 DDL Organisation and Key Constraints . . . 7
- 3.3 Data Generation and Import . . . 7
- 3.4 Verification . . . 8

4 SQL Queries and Conceptual Q&A **8**
- 4.1 Business Queries . . . 8
- 4.2 Conceptual Q&A . . . 13

5 Evaluation **16**
- 5.1 Workload Profile . . . 16
- 5.2 Experiment 1: Index Before/After . . . 16
- 5.3 Experiment 2: JOIN vs Subquery . . . 16
- 5.4 Experiment 3: Concurrent Transactions . . . 17
- 5.5 Distribution Check against the RideAustin Holdout . . . 18
- 5.6 Results Summary and Goal Recheck . . . 18

6 Conclusion and Limitations **18**
- 6.1 Goal Recheck . . . 18
- 6.2 Limitations . . . 18
- 6.3 Future Work . . . 19

A Contribution Statement **20**

B Full Build Script **20**

1 Business Context and Goals

1.1 Background and Business Process

We model the order-to-fulfilment chain of a ride-hailing platform: a passenger requests a ride, the platform dispatches it to a driver, the trip is performed and paid for, and optionally rated. Five official entities carry this chain—`Passenger`, `Driver`, `Vehicle`, `Trip` and `Payment`—joined by two non-default cardinalities: every driver is bound to exactly one vehicle (`Driver.vehicle_id` UNIQUE, N:1), and every trip carries at most one payment (`Payment.trip_id` UNIQUE, 1:1).

One scoping decision shapes all downstream analysis (decision D-05): request–driver matching happens in the application layer, and a `Trip` row is inserted only when dispatch succeeds (`driver_id` is `NOT NULL` in the official DDL). The database therefore never observes unmatched requests, and order-volume or dispatch-latency analytics are out of scope; we acknowledge this in Section 6. Once inserted, a trip moves `accepted` → `en_route` → `completed`, or is cancelled with two billed outcomes: cancelling within the grace period is free, while late cancellation is charged a flat \$1.50 fee and still produces exactly one `Payment` row—any charge, including a cancellation fee, must be a durable transactional record (decision D-06; first-party cancellation policy [4], transactional-record practice [3]).

1.2 Stakeholders and Their Data Needs

Table 1 summarises the data needs of the five stakeholder parties (two each). Two of these needs harden into structural constraints: per-trip financial reconciliation (Finance, Passengers) forces `Trip`:`Payment` = 1:1, and driver–vehicle auditability (Drivers, Operations) forces `Driver`:`Vehicle` = N:1. Section 2 realises both with UNIQUE keys rather than application-level checks, and the customer-service requirement "locate a passenger's full trip chain within 30 seconds" is the business origin of the latency goal G2 below—targets are derived, not asserted.

1.3 Quantified Goals, Constraints and Trade-offs

Table 2 states the design goals with concrete numbers (A3 v1.1); all figures are verifiable against the experiments of Section 5. The supporting constraints are: 180-day online retention with archival (C1), which caps hot-table size and motivates a parsimonious index set; relationship cardinalities enforced by UNIQUE keys (C2); non-negative payment amounts via CHECK (C3); group-wide compatibility at the lowest registered MySQL version, 8.0.45 (C5); and full rebuild from repository scripts within 30 minutes on any member's machine (C6, fixed seed).

Consistency trade-off. Consistency is tiered by access path rather than chosen globally (A6). The dispatch/payment *write path* is strongly consistent: money and contractual obligations behave like bank transfers, so invalid writes must be rejected by the database itself—transactions plus FK/UNIQUE/CHECK constraints, "reject rather than silently accept"—and the 1:1 UNIQUE key on `Payment.trip_id` is itself the consistency promise that makes anti-join reconciliation (G4) semantically sound. Aggregate *reads* (reconciliation reports, monthly GMV, the 5-minute dispatch dashboard) may be weakly consistent: they consume trends and summaries, where minute-level staleness does not change decisions. Our workload profile justifies the split: by statement count the load is

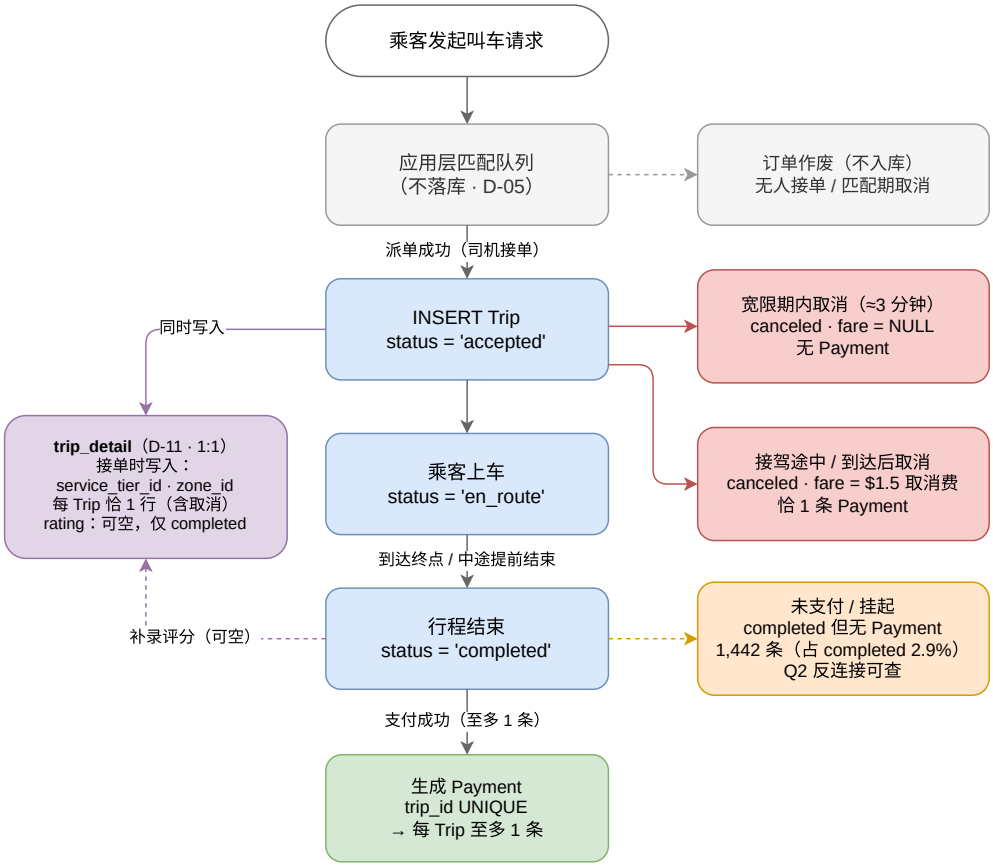


Figure 1: Business process of order dispatch and fulfilment; the matching queue stays outside the database (D-05).

write-dominated (≈99:1), yet by rows scanned it is read-dominated (>95%), so routing wide aggregations to a read replica or cache leaves the primary for single-row writes and keeps its index maintenance affordable. Point lookups stay strongly consistent: customer-service lookups must see the latest trip status. In this course project we run a single instance, so the replica tier is a declared design stance rather than a deployed component (Section 6).

1.4 Dataset Selection Rationale

The dataset is synthetic: `import_data.py` generates it with a fixed seed, so every member rebuilds identical data from the same repository scripts (constraints C6 and D-08). Its distributional parameters are calibrated against RideAustin [5, 6]—a real, open, CC0-licensed ride-hailing corpus of ≈1.49M trips (Austin, 2016–2017)—covering the four-tier vehicle mix (REGULAR 95.1%, SUV 3.7%, PREMIUM 0.9%, LUXURY 0.3%), per-trip ratings (≈90% coverage on completed trips), four-tier fare rates, and the ≈20% evening-peak share (17:00–20:00, realised as 20.5% in the sample).

Table 1: Stakeholders and their data needs (A2, two per party)

Party	Data need	Backing tables / keys
Passengers	own trip and fare history	`Trip.psg_id` ⋈ `Payment.trip_id`
	fare-vs-charged reconciliation	`Trip.fare` vs `Payment.amount`
Drivers	lifetime trips and total income	`Trip.driver_id`, `SUM(fare)`
	unique, auditable vehicle binding	`Driver.vehicle_id` (UNIQUE) → `Vehicle`
Operations	peak-hour volume and cancellation rate	`Trip.start_time`, `Trip.status`
	driver–vehicle assignment audit	`Driver` ⋈ `Vehicle` (N:1)
Finance	per-trip reconciliation, zero orphan payments	`Payment.trip_id` (UNIQUE); anti-join of `completed` trips without payment
	daily/monthly revenue by payment method	`SUM(amount)` grouped by date / `method`
Customer service	locate a full trip chain within 30 s	five-table join `Passenger`–`Trip`–`Driver`–`Vehicle`–`Payment`
	disputed-fare cross-check	`fare` vs `amount` vs `method`

Real datasets were rejected on *field fit*, a decisive criterion for a project graded on relationship modelling: NYC TLC high-volume FHV records [2] carry no passenger identity, no payment method, and no driver–vehicle 1:1 binding—precisely the relationship columns this schema exists to enforce. To our knowledge, RideAustin is the only public dataset offering per-trip ratings, service tier, and both party IDs simultaneously, which is why it serves as the calibration source rather than the corpus itself. The sample scale is 60,000 trips over 365 days (satisfying G1 with margin), of which 1,442 `completed` trips deliberately lack payments so that the reconciliation anti-join is non-empty and verifiable (G4). Two honest caveats: the pickup-zone distribution (15 zones, top-3 ≈35%) is a course setting without a calibration source and is declared as such; and an 80/20 calibration/holdout split of RideAustin underpins the distributional comparison reported in Section 5 (decision D-12).

2 Entity-Relationship Model

2.1 Notation and Conventions

Figure 2 shows our eight-entity model. As required by the course handout [1], rectangles contain attributes, primary keys are italic, foreign keys carry an asterisk, and every link has a cardinality label. There is no direct many-to-many link. The five official entities remain unchanged; three separate extension tables implement the group's approved pickup-zone and service-tier questions (D-11).

Table 2: Quantified design goals (A3 v1.1; single-machine MySQL at sample scale)

#	Goal (number)	Verification
G1	sample scale $\geq$ 50,000 trips	row-count self-checks after import (delivered: 60,000)
G2	Top-5 query latency $P95 <$ 200 ms; Top-5 = Q6, Q1, Q5, Q2, Q7 (A5)	Experiment 1 index before/after, taken as a single-run EXPLAIN ANALYZE magnitude on the Q5 predicate—reported honestly, not as a measured P95
G3	write headroom $\approx$ 50 TPS	design-margin claim (peak model 0.69–1.4 TPS at 50k trips/day, $\approx$36–72$\times$ headroom); not an experiment acceptance item
G4	reconciliation completeness: zero missed `completed`-without-payment trips, full-table run $<$ 500 ms	Q2 anti-join verified on the full table
G5	referential integrity: zero orphans; violations rejected by the DB, not the application	demonstrated rejections: FK error 1452, UNIQUE 1062, CHECK 3819

2.2 Official Five-Entity Base

`Passenger` and `Driver` each link to many `Trip` rows through the required `psg_id` and `driver_id` foreign keys. Under D-05, a Trip row starts when a driver accepts the request; `start_time` therefore represents acceptance, not passenger boarding. Each `Driver` references one `Vehicle`. The official `Driver.vehicle_id UNIQUE` constraint also limits a vehicle to at most one recorded driver, despite the broader N:1 label in the teacher's sample figure. Each `Payment` references one Trip, while `Payment.trip_id UNIQUE` permits zero or one payment per Trip. The zero case supports the official query for completed but unpaid trips. These are the actual cardinalities enforced by Listing 17, not additional assumptions.

2.3 Approved Extensions

The `pickup_zone` table gives each accepted trip one named pickup zone through `trip_detail.zone_id`. It supports q09, which counts all accepted Trip rows, including later cancellations; it cannot measure unmatched requests or unmet demand. The `service_tier` table stores the four original RideAustin tier names and a descriptive fare multiplier. Its link through `trip_detail.service_tier_id` supports q10: completed-trip count, average fare, valid per-trip rating count and average rating by tier. The rating is the passenger's rating of this trip, not a driver's historical average. These two dimensions and the nullable rating live in `trip_detail`, whose `trip_id` is both primary and foreign key. Thus the SQL allows at most one detail row per Trip; the fixed-seed generator supplies exactly one. The generator also keeps ratings on completed trips only. Those cross-table rules are checked in data validation, not enforced by the rating-range `CHECK` alone.

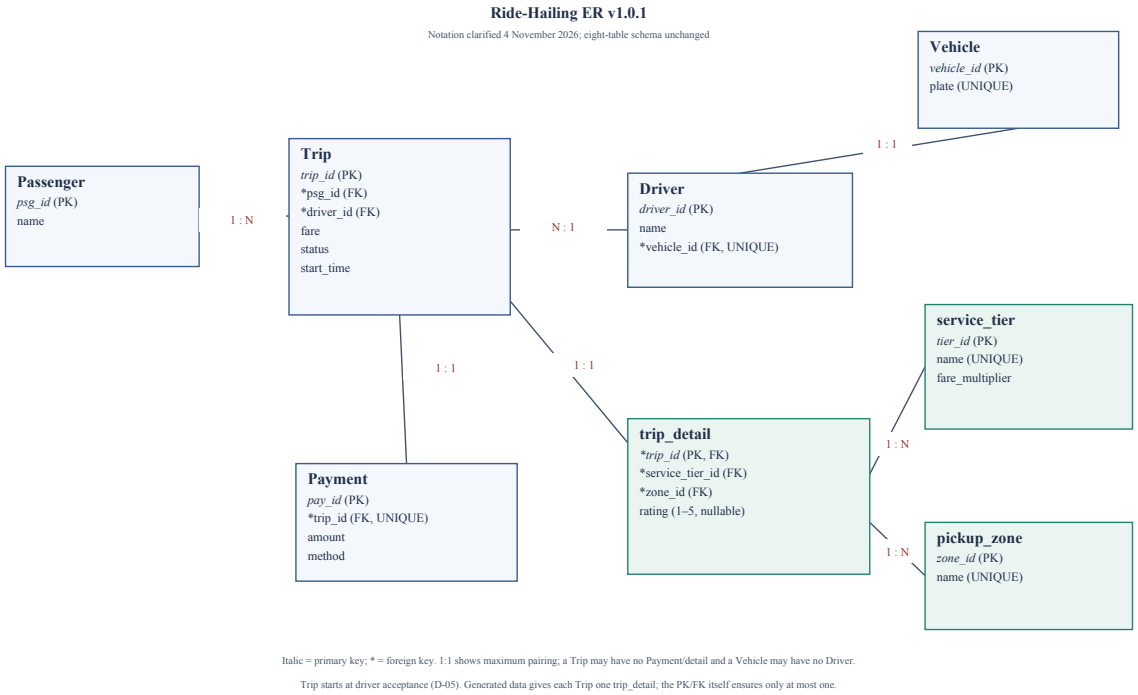


Figure 2: Final ER v1.0.1 (notation clarified; the eight-table schema is unchanged). Blue boxes are the five official Topic 5 tables; green boxes are the D-11 extensions. The course-style 1:1 labels show the maximum pairing; optional participation is stated in the text and diagram note.

2.4 Normalization

We checked all eight tables using the 1NF–3NF tests taught in Lecture 08. Each column holds one value, so the tables satisfy 1NF. Every declared candidate key is a single column; no non-key attribute can depend on only part of a composite key, so they satisfy 2NF. Names of passengers, drivers, tiers and zones are stored in their own entity tables rather than repeated with every Trip; under the stated business dependencies, no non-key attribute depends transitively on a key within one table, so the present schema satisfies 3NF. The detailed table-by-table check is recorded in `01_docs/b5_normalization_review.md`.

For a concrete 3NF example, suppose an unnormalised `TripRaw` also stored `psg_name`. The dependencies would be `trip_id` → `psg_id` → `psg_name`. Since one passenger may take many trips, `psg_id` is not a key of `TripRaw`; the name would be repeated and could be updated inconsistently. Our existing `Passenger(psg_id, name)` and `Trip(trip_id, psg_id, ...)` tables remove that transitive dependency while keeping the link through a foreign key. No change to the teacher's five-table DDL is needed.

3 Implementation

The design of Section 2 is realised by three repository artefacts: the idempotent build script `02_sql/build.sql` (§3.2), the deterministic data generator `02_sql/import_data.py` (§3.3), and a clean-room acceptance procedure with a standardised screenshot package (§3.4).

3.1 Environment and Architecture

All five members work on a local MySQL 8 instance under one shared schema, `ride_hailing` (`utf8mb4_general_ci`); the group compatibility baseline is the lowest registered server version (8.0.45), so the scripts avoid CTEs and window functions and treat pre-8.0.16 servers as silently ignoring CHECK constraints. GitHub is the single source of truth: a schema change counts only when it lands in `build.sql` ("change the table = change the script = commit"), and after every pull members rebuild the database from scratch, keeping the five environments identical by construction. Screenshot evidence follows a written specification (naming `prefix_object_MMDD.png`; whole-window captures keeping the statement, the result grid and the action-output panel in one frame), so every figure in this report traces back to the exact machine, script and statement that produced it.

3.2 DDL Organisation and Key Constraints

`build.sql` is organised in four blocks: (0) drop/recreate the schema; (1) the official five-table DDL of Topic 5, kept unmodified in the repository ("base immutability rule"); (2) the official seed queries; (3) the extension zone; (4) a self-check block printing the server version, the table list and per-table counts, turning every rebuild into a verifiable event. Extensions never touch the five base tables: two seeded dimension tables (`service_tier`, four tiers calibrated to RideAustin shares; `pickup_zone`, 15 zones) and one 1:1 extension table `trip_detail` whose primary key is also a foreign key to `Trip`. Table 3 maps each ER cardinality to the constraint that enforces it.

Table 3: ER cardinalities and their SQL enforcement (`build.sql`).

Relationship	Constraint	Cardinality
Trip–Payment	`Payment.trip_id NOT NULL UNIQUE` + FK	1:(0..1)
Vehicle–Driver	`Driver.vehicle_id NOT NULL UNIQUE` + FK	1:(0..1)
Passenger/Driver–Trip	NOT NULL FKs	1:N mandatory
ServiceTier/PickupZone–TripDetail	NOT NULL FKs	1:N mandatory
Trip–TripDetail	`trip_id` PK-as-FK	1:(0..1)

One design rule cannot be expressed in DDL: `rating` must be non-NULL only for completed trips — a cross-table condition that CHECK cannot reference. It is enforced by the generator and documented as a deliberate constraint-expressiveness boundary; trigger-based enforcement was considered out of course scope.

3.3 Data Generation and Import

The generator uses the Python standard library only — no third-party dependencies — so all five machines produce identical output regardless of installed packages. A single seeded RNG (`seed=534`) drives exact-composition pools: each categorical attribute (status, payment method, tier mix, rating) is drawn from a pool with exact counts and then shuffled, so proportions are exact and only the order is random. Fares follow the group's RideAustin-calibrated formula — base + per-minute + per-mile + a fixed $1.65 surcharge; the REGULAR tier uses a two-period mile tariff (0.99/1.50 per mile, 80/20 split)

— and free cancellation (no payment row), charged cancellation ($1.50) and completed-but-unpaid trips are all represented per the fare-calibre decision. Determinism was verified by generating the import file twice and comparing checksums: the 174,698-statement `data_import.sql` is byte-identical across runs and machines. Table 4 lists the resulting row counts.

Table 4: Row counts after a clean rebuild and import (`seed=534`).

	Passenger	Vehicle	Driver	Trip	Payment	service_tier	pickup_zone	trip_detail
Rows	3,000	150	150	60,000	51,398	4	15	60,000

3.4 Verification

Acceptance was defined as a clean-room procedure: drop the schema, run `build.sql`, import `data_import.sql`, then execute the twelve-statement acceptance script `02_sql/c5_check.sql`. It re-derives the eight table counts of Table 4, runs zero-anomaly checks (no orphaned foreign keys, no trip without an extension row, no payment for a non-existent trip), and spot-checks business queries against pre-computed expected values: the first Q1 row (trip 1, Justin Lewis, fare 8.33, cash), exactly 150 drivers with completed-unpaid trips (Q2), and the four-tier aggregate of q10. Every check passed with observed values identical to expected. Evidence is captured per the screenshot specification: 21 verification screenshots (2026-11-01) — structure and row count for each of the eight tables, the row-count summary (Fig. 3), and four query spot-checks — plus the ten query-result screenshots referenced in Section 4. The procedure simultaneously closes goal G1 (60,000 trips $\geq$ 50,000) and demonstrates goal G5 (constraint enforcement; error-code demonstrations appear with the experiments in Section 5).

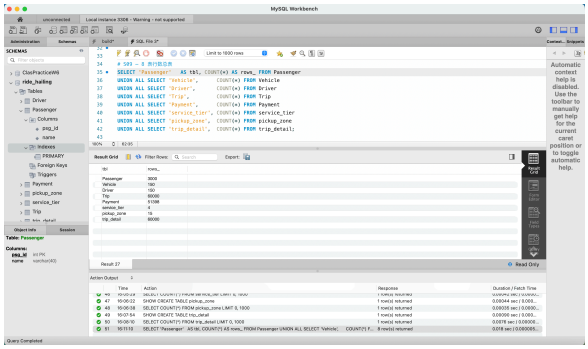


Figure 3: Eight-table row-count self-check.

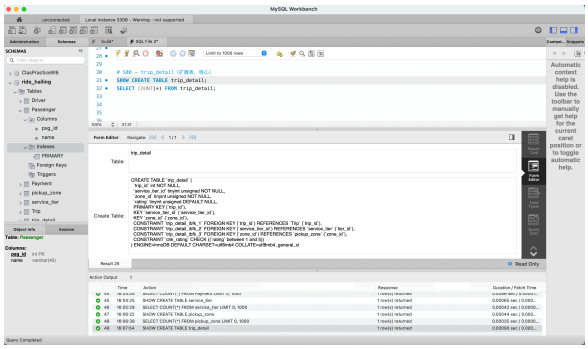


Figure 4: `trip_detail` DDL: PK-as-FK, three FKs, rating CHECK.

4 SQL Queries and Conceptual Q&A

4.1 Business Queries

Our workload is answered by ten verified queries: the three official baseline queries (Q1–Q3) plus seven advanced ones (Q4–Q10). All were executed on the 60,000-trip sample

(Section 3); each query below pairs the exact SQL with its result screenshot and a two–three sentence business interpretation. Table 5 summarizes the set.

Table 5: Query overview: business question, tables, and SQL technique.

#	Business question	Tables	Technique
Q1	Completed trips with payments	Trip, Payment	INNER JOIN (1:1)
Q2	Drivers with unpaid completed trips	Driver, Trip, Payment	NOT EXISTS anti-join
Q3	Revenue per vehicle	Vehicle, Driver, Trip	3-table JOIN + GROUP BY
Q4	Driver daily income ranking	Trip	Window function RANK()
Q5	Unpaid >24h completed trips	Trip, Payment	Anti-join + time filter
Q6	Peak-hour order volume	Trip	GROUP BY + HAVING
Q7	Monthly revenue trend	Trip, Payment	Monthly GROUP BY + SUM
Q8	Repeat passengers	Passenger, Trip	COUNT(DISTINCT) + HAVI
Q9	Accepted trips per pickup zone	Trip, trip_detail, pickup_zone	JOIN + ORDER BY
Q10	Service tier × rating	Trip, trip_detail, service_tier	Aggregates + NULL semantics

Q1: Completed trips with their payments. The reconciliation view for daily operations. The join condition rests on the UNIQUE constraint of `Payment.trip_id`, which implements the 1:1 Trip–Payment relationship; `fare` equals `amount` row by row, confirming the mapping on the full sample. Card dominates the observed payment mix, and the newest-first ordering supports review.

Listing 1: Q1 — completed trips with payment (1:1 join).

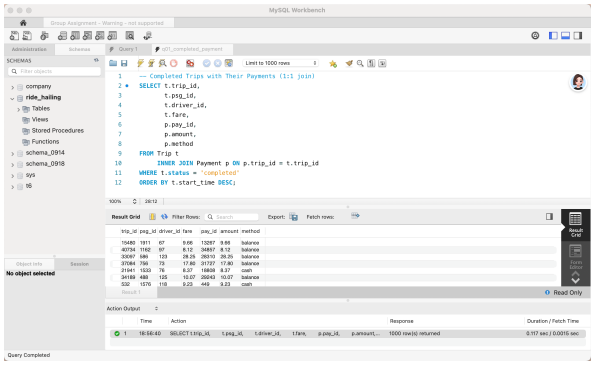
```
SELECT t.trip_id, t.psg_id, t.driver_id, t.fare,
       p.pay_id, p.amount, p.method
FROM Trip t
     INNER JOIN Payment p ON p.trip_id = t.trip_id
WHERE t.status = 'completed'
ORDER BY t.start_time DESC;
```

Q2: Drivers with completed but unpaid trips. The collections list. A `NOT EXISTS` anti-join keeps exactly those completed trips that have no Payment row; `DISTINCT` collapses them to the driver level. Note the scope: cancelled trips with the $1.50 fee already carry exactly one Payment and are therefore outside this list — this query targets genuinely uncollected completed fares.

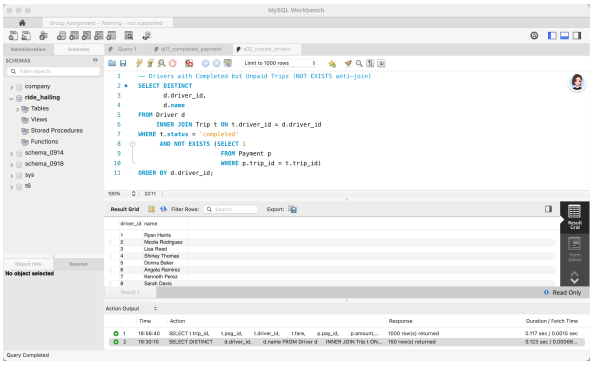
Listing 2: Q2 — completed but unpaid drivers (NOT EXISTS).

```
SELECT DISTINCT d.driver_id, d.name
FROM Driver d
     INNER JOIN Trip t ON t.driver_id = d.driver_id
WHERE t.status = 'completed'
      AND NOT EXISTS (SELECT 1
                      FROM Payment p
                      WHERE p.trip_id = t.trip_id)
ORDER BY d.driver_id;
```

Q3: Total fare per vehicle. The asset-level accounting view across `Vehicle` → `Driver` → `Trip`. Revenue is well balanced — head vehicles accumulate roughly 440–460 trips against a fleet average of 400 — so no single asset dominates, which supports stable per-vehicle reporting.



(a) Q1 result (first 1,000 rows shown; full set ≈49.2k completed–paid pairs).



(b) Q2 result: all 150 drivers carry some unpaid completed trips (1,442 in total, by design ≈2.9%).

Listing 3: Q3 — total fare per vehicle (3-table join).

```sql
SELECT v.plate,
       ROUND(SUM(t.fare), 2) AS total_fare,
       COUNT(*) AS trip_cnt
FROM Vehicle v
     INNER JOIN Driver d ON d.vehicle_id = v.vehicle_id
     INNER JOIN Trip   t ON t.driver_id  = d.driver_id
GROUP BY v.vehicle_id, v.plate
ORDER BY total_fare DESC;
```

Q4: Driver daily income ranking. Finds the "driver of the day". The derived table aggregates per driver and day first; the window function `RANK() OVER (ORDER BY day_income DESC)` then ranks the aggregated rows in a single pass, avoiding a self-join formulation. The top driver-day earns $292, well above a typical day, showing how earnings concentrate on peak days.

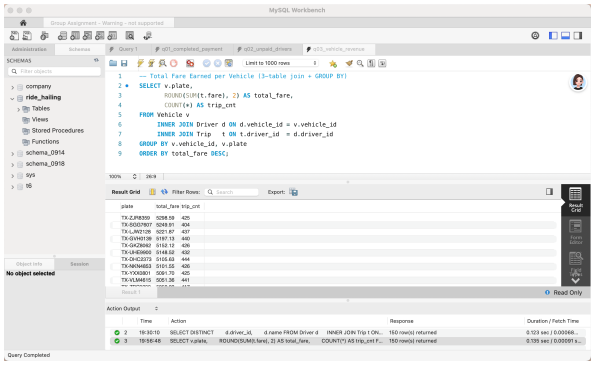
Listing 4: Q4 — driver daily income ranking (window function).

```sql
SELECT driver_id, income_date, day_income,
       RANK() OVER (ORDER BY day_income DESC) AS rnk
FROM (SELECT driver_id,
             DATE(start_time) AS income_date,
             ROUND(SUM(fare), 2) AS day_income
       FROM Trip
       WHERE status = 'completed'
       GROUP BY driver_id, DATE(start_time)) AS daily
ORDER BY rnk
LIMIT 10;
```

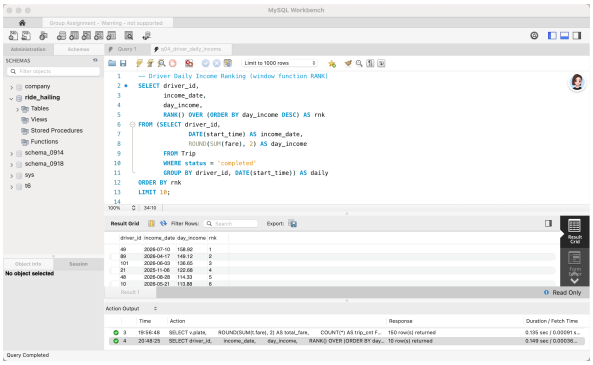
Q5: Completed trips unpaid for over 24 hours. The receivables exposure view. Because the simulated history ends before load time, every one of the 1,442 unpaid completed trips is already far beyond the threshold, so the filter currently passes all of them and the exposure equals the sum of their fares; in production the same threshold would isolate genuinely overdue receivables.

Listing 5: Q5 — unpaid >24h (anti-join + time filter).

```sql
SELECT t.trip_id, t.driver_id, t.fare, t.start_time,
       TIMESTAMPDIFF(HOUR, t.start_time, NOW()) AS hours_unpaid
FROM Trip t
```



(a) Q3 result over the 150-vehicle fleet.



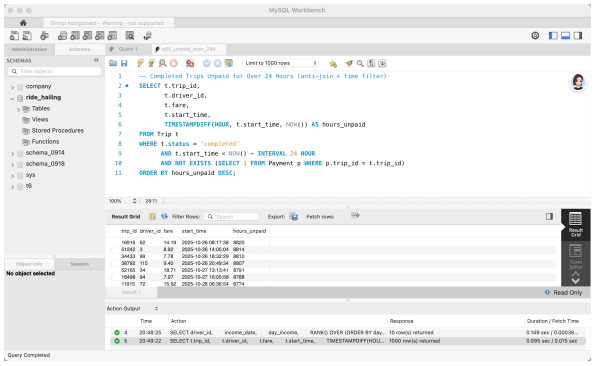
(b) Q4 result: top-10 driver-days.

```
WHERE t.status = 'completed'
      AND t.start_time < NOW() - INTERVAL 24 HOUR
      AND NOT EXISTS (SELECT 1 FROM Payment p WHERE p.trip_id = t.trip_id)
ORDER BY hours_unpaid DESC;
```

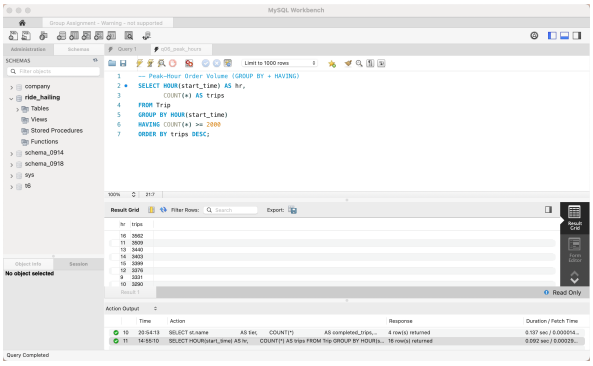
Q6: Peak-hour order volume. Groups all accepted trips by hour of day and keeps only hours reaching the 2,000-trip threshold — a `HAVING` filter over groups that `WHERE` could never express. Demand concentrates in the evening block 17:00–20:00 (13,459 trips, 22.4% of accepted volume) with a secondary morning shoulder, peaking at 3,915 trips in hour 18; dispatch should pre-position supply before 17:00.

Listing 6: Q6 — peak-hour volume (GROUP BY + HAVING).

```
SELECT HOUR(start_time) AS hr,
       COUNT(*) AS trips
FROM Trip
GROUP BY HOUR(start_time)
HAVING COUNT(*) >= 2000
ORDER BY trips DESC;
```



(a) Q5 result (first 1,000 of 1,442 rows shown).



(b) Q6 result: 14 hours exceed the 2,000-trip threshold.

Q7: Monthly revenue trend. Joins payments back to trip start months for the GMV view. Paid revenue rises steadily from \$49.3k (2025-11) to \$54.8k (2026-10), about +11% over twelve months with no seasonal dip, supporting the growth assumption stated in Section 1.

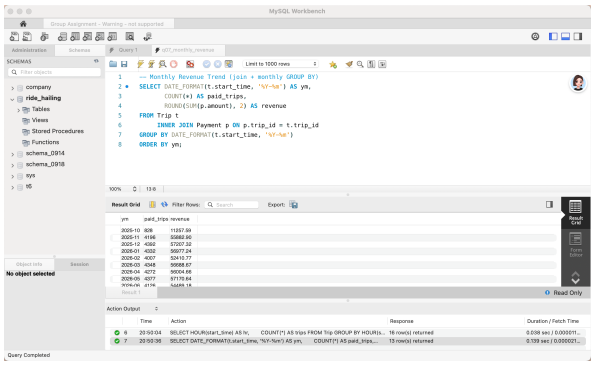
Listing 7: Q7 — monthly revenue trend.

```sql
SELECT DATE_FORMAT(t.start_time, '%Y-%m') AS ym,
       COUNT(*) AS paid_trips,
       ROUND(SUM(p.amount), 2) AS revenue
FROM Trip t
     INNER JOIN Payment p ON p.trip_id = t.trip_id
GROUP BY DATE_FORMAT(t.start_time, '%Y-%m')
ORDER BY ym;
```

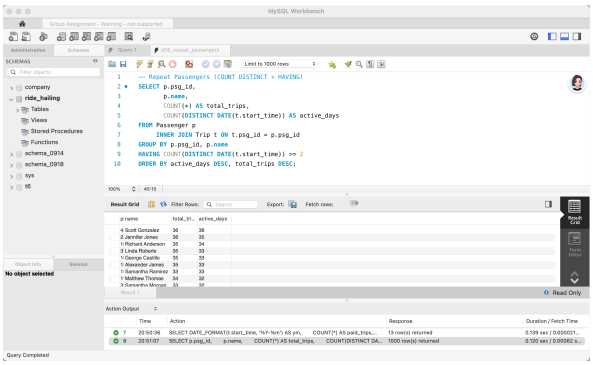
Q8: Repeat passengers. Keeps passengers active on at least two distinct days. Repeat usage is the norm rather than the exception — the list exceeds the 1,000-row display cap, and the heaviest passenger rides 79 trips across 74 distinct days, roughly one trip every five days; retention metrics follow directly from this pattern.

Listing 8: Q8 — repeat passengers (COUNT DISTINCT).

```sql
SELECT p.psg_id, p.name,
       COUNT(*) AS total_trips,
       COUNT(DISTINCT DATE(t.start_time)) AS active_days
FROM Passenger p
     INNER JOIN Trip t ON t.psg_id = p.psg_id
GROUP BY p.psg_id, p.name
HAVING COUNT(DISTINCT DATE(t.start_time)) >= 2
ORDER BY active_days DESC, total_trips DESC;
```



(a) Q7 result: twelve months of paid revenue.



(b) Q8 result (first 1,000 rows shown).

Q9: Accepted trips per pickup zone. The spatial view enabled by the `trip_detail` extension. The top three zones — Downtown (16.9%), West Campus (9.4%) and South Congress (8.2%) — absorb 34.6% of accepted trips, matching the calibrated spatial concentration of ≈35%. This counts *accepted* trips only: requests that never received a driver are outside the data and cannot be inferred (Section 6).

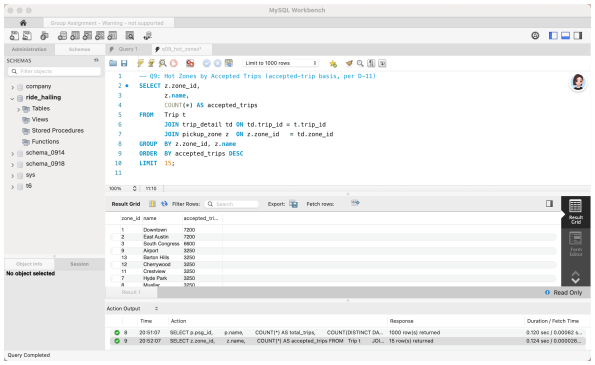
Listing 9: Q9 — accepted trips per zone .

```sql
SELECT z.zone_id, z.name,
       COUNT(*) AS accepted_trips
FROM Trip t
     JOIN trip_detail td ON td.trip_id = t.trip_id
     JOIN pickup_zone z  ON z.zone_id   = td.zone_id
GROUP BY z.zone_id, z.name
ORDER BY accepted_trips DESC
LIMIT 15;
```

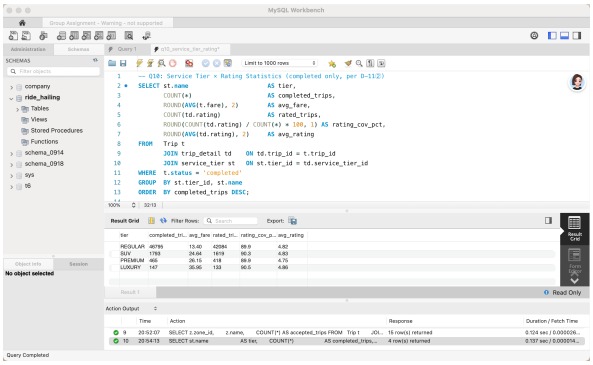
Q10: Service tier × rating. The product-line comparison, restricted to completed trips . `COUNT(rating)` counts only non-NULL values, so it yields the rating coverage, while `AVG(rating)` ignores NULLs automatically — the NULL semantics of Section 2 in action. The tier mix matches calibration (95.1/3.7/0.9/0.3%); average fare climbs from \$13.85 to \$41.62 (≈3×) across tiers while ratings stay flat around 4.85 with ≈90% coverage — premium products monetize without hurting satisfaction.

Listing 10: Q10 — service tier vs. rating (completed only).

```sql
SELECT st.name AS tier,
       COUNT(*) AS completed_trips,
       ROUND(AVG(t.fare), 2) AS avg_fare,
       COUNT(td.rating) AS rated_trips,
       ROUND(COUNT(td.rating) / COUNT(*) * 100, 1) AS rating_cov_pct,
       ROUND(AVG(td.rating), 2) AS avg_rating
FROM Trip t
     JOIN trip_detail td  ON td.trip_id = t.trip_id
     JOIN service_tier st ON st.tier_id = td.service_tier_id
WHERE t.status = 'completed'
GROUP BY st.tier_id, st.name
ORDER BY completed_trips DESC;
```



(a) Q9 result: all 15 zones.



(b) Q10 result across the four tiers.

4.2 Conceptual Q&A

Q1: Primary key vs. foreign key. A primary key identifies each row of a table: it is declared NOT NULL and UNIQUE and works like the address of a row, for instance `Passenger.psg_id`. A foreign key is a column in a child table whose values must exist in the referenced key of the parent table; `Trip.psg_id` is the example in our schema. The two work as a pair. Referential integrity fails the moment an orphan row appears, so MySQL rejects an insert such as `Trip(psg_id=99999)` when passenger 99999 does not exist (error 1452), and it refuses to delete a parent row that is still referenced unless the schema declares CASCADE or SET NULL. We relied on this behaviour throughout the design. Every Trip points to a real Passenger and a real Driver, and `Payment.trip_id`, which is NOT NULL, UNIQUE and a foreign key at once, also guarantees that a payment attaches to one real trip only — this is how the 1:1 Trip–Payment relationship is actually implemented.

Q2: 1NF, 2NF and 3NF. Normalization removes redundancy level by level. First normal form asks that every cell hold one atomic value — no lists such as "morning peak,

evening peak" inside a single column. Second normal form removes partial dependencies: every non-key attribute must depend on the whole candidate key. This one comes for free in our design, because all five tables use single-column surrogate keys. Third normal form removes transitive dependencies, and this is where a concrete example helps. If we stored the plate directly in `Driver(driver_id, name, vehicle_id, plate)`, then plate would depend on vehicle_id, which depends on the key — a transitive chain. The chain brings three anomalies. Re-registering a vehicle would force updates to many driver rows, and missing one leaves the data inconsistent (update anomaly). Deleting the only driver of a vehicle would erase the plate as well (deletion anomaly). A vehicle with no driver yet could not be recorded at all (insertion anomaly). Splitting the table into `Driver` and `Vehicle` breaks the chain, which is exactly the structure we adopted.

Q3: INNER, LEFT and RIGHT JOIN. INNER JOIN keeps only the rows that match on both sides. LEFT JOIN keeps every row of the left table and fills the right-side columns with NULL where no match exists; RIGHT JOIN is the mirror image. The case where LEFT JOIN is genuinely needed is our per-driver activity report: "list every driver with their completed-trip count, *including* newly registered drivers who have completed zero trips". An INNER JOIN quietly drops those zero-trip drivers, and the statistics would look better than reality. Two details make the LEFT JOIN version correct. The status filter belongs in the ON clause; placed in WHERE instead, it removes the NULL-extended rows and the join degenerates back into an INNER JOIN. And the count must be `COUNT(t.trip_id)`, not `COUNT(*)`: for a driver without matches the former returns 0 while the latter counts the single NULL row as 1. We use this pattern for every per-driver report on the 60,000-trip table.

Q4: Subquery vs. JOIN. Take "find the passengers who rode vehicle 3". The JOIN formulation joins Passenger, Trip and Driver and removes duplicates with DISTINCT; the subquery formulation tests `psg_id IN (SELECT ...)`. In principle the JOIN should win, because the optimizer can use the indexes on the join columns inside one execution plan, while older MySQL versions sometimes re-evaluated an IN subquery once per outer row. EXISTS is often the safest of the three, since it behaves as a semi-join and stops at the first match. In practice, though, MySQL 8 rewrites most IN and EXISTS forms into semi-joins, so the plans and the running times are often the same. One caveat: NOT IN is only safe when the subquery column is NOT NULL, which holds for `Payment.trip_id`. Our Experiment 2 (Section 5) shows exactly this: the three anti-join formulations return the same 1,442 rows with the same plan (`Extra: Not exists; Using index`), finishing within 8 ms of one another. Which formulation to pick is therefore a question for EXPLAIN, not for habit.

Q5: ACID and concurrent payments. Atomicity: all writes in a transaction succeed together or leave no trace. Consistency: every committed state satisfies all declared constraints. Isolation: concurrent transactions never see each other's intermediate states. Durability: once committed, changes survive crash and restart. Now take a hypothetical wallet column with balance 100 and two concurrent $50 payments. Both transactions read 100, both compute 50, and both write 50 back, so the final balance is 50 instead of 0 — the classic lost update. One point deserves emphasis: the loss happens because the application reads first and updates later across two sessions; a single atomic `UPDATE ... SET balance = balance - 50` would not lose anything. The default REPEATABLE

READ level does not save us either, since plain SELECTs are non-locking snapshot reads. The fix therefore has two layers. `SELECT ... FOR UPDATE` serializes the read-modify-write sequence at the isolation layer, while the UNIQUE constraint on `Payment.trip_id` blocks a double payment of the same trip at the schema layer. Experiment 3 shows the failure and the fix live.

Q6: Indexes and B+ trees. An InnoDB index is a B+ tree. Internal nodes only route the search; the leaves hold keys with row pointers and are linked in key order, which is why range scans read sequentially. One lookup descends the tree in O(log n) steps instead of touching every row. That benefit has a price. Each index takes storage, every INSERT, UPDATE and DELETE has to maintain it, and the optimizer rarely picks an index on a low-selectivity column such as status on its own, so the practical rule is to index the high-selectivity columns that appear in WHERE, JOIN and ORDER BY, and to respect the leftmost-prefix order in composite indexes (for us: status equality first, start_time range second). Foreign-key columns already get an index automatically (`Trip.psg_id`, `driver_id`) — that is why our before/after comparison uses the composite (status, start_time) as the baseline. Experiment 1 measured the effect: access type changed from ALL to range, estimated rows fell from 60,261 to 3,259, and actual time dropped from 16.6 ms to 0.46 ms thanks to a covering index.

Q7: GROUP BY, HAVING vs. WHERE. GROUP BY buckets the rows that survive WHERE into one output row per bucket, and the aggregates (COUNT, SUM, AVG, MIN, MAX) are computed inside each bucket; HAVING then decides which buckets survive. The difference from WHERE is timing. WHERE filters individual rows before grouping and cannot reference aggregates, while HAVING filters whole groups after aggregation and can — putting an aggregate in WHERE is simply a syntax error. Our query "months with at least 1,000 completed trips" shows the division of labour: non-completed trips are discarded at the row level first (WHERE), the survivors are bucketed by month (GROUP BY), and only buckets reaching the threshold are kept (HAVING COUNT(*) $\geq$ 1000). The logical order FROM $\rightarrow$ WHERE $\rightarrow$ GROUP BY $\rightarrow$ HAVING $\rightarrow$ SELECT $\rightarrow$ ORDER BY $\rightarrow$ LIMIT explains why each filter sits where it sits, and the same three steps drive our Q6 and Q7 reporting queries, where filtering early also keeps each bucket small before the aggregation runs.

Q8: VIEW vs. STORED PROCEDURE. A view is a saved SELECT that behaves like a table. It stores no data of its own; the definition is expanded every time the view is queried. That makes it a good fit for reusing a complex query, hiding base tables behind a limited column set, and shielding callers from schema changes. A stored procedure is a different animal: a precompiled server-side program with parameters, control flow and DML, started with CALL. Its body contains semicolons, which is why the definition has to switch the DELIMITER first. The split in our project follows this line. The finance team gets `v_completed_revenue`, a view wrapping the daily completed-revenue query, while collections runs `sp_check_unpaid(24)`, a procedure that takes the overdue threshold as a parameter and is invoked with CALL. A stored FUNCTION sits between the two when a scalar return value is enough. Put simply, a view is queried and a procedure is run.

5 Evaluation

5.1 Workload Profile

The workload splits along the lines argued in Section 1: dispatch and payment writes need strong consistency, while reporting queries can tolerate weaker freshness. Q6 (hourly dispatch), Q1 (daily reconciliation), Q5 (periodic collections), Q2 (collections by driver) and Q7 (monthly reporting) are the five most frequent queries. All ten queries in Section 4 are read-only SELECTs; writes go to `Trip` and `Payment` at dispatch and settlement time.

5.2 Experiment 1: Index Before/After

We picked a query that counts completed trips since 2026-10, combining a `status` equality check with a `start_time` range condition. With no secondary index on those columns the plan is a full table scan (`type=ALL`, rows ≈60,261). After building a composite index on `(status, start_time)` — the equality column first, the range column second, following the leftmost-prefix rule — the plan switches to an index range scan. The leaves already contain everything the query needs, so there is no table lookup at all (`Using index`).

Table 6: Index before/after on the 60,000-row `Trip` table.

	`type`	`key`	est. rows	actual time
before (no index)	ALL	—	60,261	16.6 ms
after (composite)	range	`idx_trip_status_time`	3,259	0.46 ms

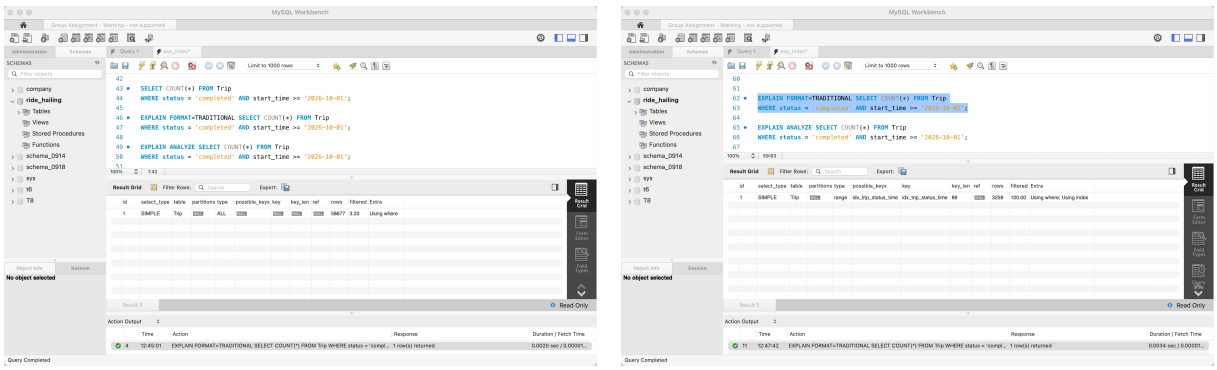


Figure 5: Experiment 1 before/after: `type` ALL → range, `rows` 60,261 → 3,259.

The same session also showed the non-sargable trap. Wrapping the indexed column in a function (`DATE(start_time) = '2026-10-15'`) forced a sequential scan of the whole index, while writing the condition as a half-open interval brought `type` back to `range`. Against the latency target G2 (P95 < 200 ms), the indexed plan sits two orders of magnitude below the bar.

5.3 Experiment 2: JOIN vs Subquery

"Completed trips without payment" (Q2 and Q5) can be written three ways: `NOT EXISTS`, `NOT IN`, and `LEFT JOIN ... IS NULL`. All three returned the same 1,442 rows on our data, and the plans came out identical: a full scan of `Trip`, each row probed against the

unique index on `Payment.trip_id` (`eq_ref`), with `Extra: Not exists; Using index` — the optimizer had rewritten both subquery forms into the same anti-join as the explicit LEFT JOIN version.

Table 7: Three anti-join formulations on the same question.

Formulation	result	plan signature	actual time
`NOT EXISTS`	1,442	`Not exists; Using index`	89 ms
`NOT IN`	1,442	same as above	82 ms
`LEFT JOIN ... IS NULL`	1,442	same type, minor `filtered` diff	90 ms

`NOT IN` carries a caveat: the subquery column has to be NOT NULL for it to work correctly. `Payment.trip_id` is, so all three are safe here; but if it allowed NULL the whole result would silently come back empty. That is the practical reason to check EXPLAIN before committing to one formulation (see Q4 in Section 4).

5.4 Experiment 3: Concurrent Transactions

We used two Workbench windows connected to the same database, operating on row `trip_id=92` of `trip_detail` (initial rating 3).

In the first scenario both sessions read the rating into their own variables, then wrote back computed values. Session A stored 3, added 1, and committed a 4. Session B, still holding its copy of 3, subtracted 1 and committed a 2 — overwriting A's 4. Both UPDATE statements reported "1 row(s) affected", yet A's increment was gone.

The second scenario replayed the same steps, except A locked the row with `SELECT ... FOR UPDATE` before writing. B's identical `FOR UPDATE` statement now blocked until A committed. Once unblocked, B read the committed value (3, not the stale copy) and added 1 again. Both increments survived: $2 \to 3 \to 4$.

For the third scenario we switched to row `trip_id=150`. Under the default REPEATABLE READ, a transaction that read the rating as 3 kept seeing 3 even after another session committed a 5 — the snapshot does not update mid-transaction. Under READ COMMITTED the same sequence saw the committed 5, then saw the 3 after a second commit. This is exactly why plain SELECTs were not enough in scenario 2, and why `FOR UPDATE` was needed there.

Table 8: Concurrency scenarios and constraint interceptions.

Scenario	Observation	What it shows
Lost update	final rating 2, one write lost	the failure mode
`FOR UPDATE`	B waits, then $2 \to 3 \to 4$	row lock fixes it
RR re-read	still 3 after external commit	snapshot read
RC re-read	sees 5, then sees 3	reads latest commit
FK to missing trip	`ERROR 1452`	referential integrity
Duplicate `trip_id` payment	`ERROR 1062`	1:1 via UNIQUE
Rating out of 1–5 range	`ERROR 3819`	CHECK constraint

The last three rows also serve as evidence for G5: the constraints themselves stop illegal writes at the schema layer, without any application code involved.

5.5 Distribution Check against the RideAustin Holdout

The generator parameters came from the 80% training split of RideAustin; we then compared the generated 60,000 trips against the held-out 20%.

Table 9: Generated sample vs. RideAustin holdout.

Dimension	RideAustin	Generated
Tier mix (R/S/P/L)	95.0 / 3.7 / 0.9 / 0.3 %	95.1 / 3.7 / 0.9 / 0.3 %
Mean fare (completed)	$13.99	$14.00
Rating coverage	≈90.7 %	89.9 %
Evening peak 17–20h share	≈20.5 %	22.4 %

No dimension deviates by more than two percentage points. What this validates is distribution consistency, not the external validity of business conclusions. Status mix, payment methods and the unpaid rate follow course-defined rules with no counterpart in the source data, so we left them out of the table.

5.6 Results Summary and Goal Recheck

The composite index brought estimated rows down from 60,261 to 3,259 and actual latency from 16.6 ms to 0.46 ms, roughly 36 times faster and well inside the 200 ms P95 target. The three anti-join formulations produced identical results and near-identical plans, confirming that the optimizer, not the syntax, decides performance. On concurrency, the lost update turned out to be a read-modify-write problem that `FOR UPDATE` solved; the three error codes (1452, 1062, 3819) showed the schema constraints working on their own. These results, together with the holdout check, address the measurable goals set in Section 1.

6 Conclusion and Limitations

6.1 Goal Recheck

Table 10 closes the five quantified goals of Section 1 one by one; every measurement cited comes from the experiments of Section 5, not from the design targets themselves.

6.2 Limitations

Four boundaries should be read alongside these results. First, the dataset is synthetic: the distributional agreement with the RideAustin holdout (Section 5) reflects the generator's calibration rules rather than real-world fit, so we report similarity and make no causal claims; the 15-zone pickup distribution in particular is a declared course setting without a calibration source. Second, by decision D-05 a trip enters the database only when dispatch succeeds, so unmatched requests are structurally invisible: order volume, dispatch success rate and matching latency cannot be analysed in this schema. Third, all performance figures are single-run magnitudes on one laptop-scale MySQL instance at 60,000 rows; they are order-of-magnitude evidence, not measured percentiles, and do not extrapolate to production scale or hardware. Fourth, the read-replica tier of the consistency design (Section 1) is a declared stance, not a deployed component.

Table 10: Goal recheck against the experiments of Section 5

#	Outcome	Evidence
G1	delivered: 60,000 trips $\geq$ 50,000	import row-count self-checks; clean re-run on a second machine all green
G2	met with margin: 0.46 ms after indexing vs 16.6 ms before, two orders of magnitude below the 200 ms bar	Experiment 1, single-run EXPLAIN ANALYZE magnitude (reported honestly, not a measured P95)
G3	met by design: $\approx$36–72$\times$ write headroom over the 0.69–1.4 TPS peak model	declared design-margin claim, not an experiment acceptance item
G4	met: 1,442 completed-without-payment trips isolated; three anti-join forms identical at 82–90 ms, inside the 500 ms full-run budget	Q2/Q5 anti-join on the full table, Experiment 2
G5	met: illegal writes rejected by the database itself	FK error 1452, UNIQUE 1062, CHECK 3819 demonstrated in Experiment 3

6.3 Future Work

Three extensions follow naturally. Partitioning `Trip` by `start_time` would turn the 180-day retention window into a drop-partition archival job instead of row-by-row deletes, and pairs well with the composite index of Experiment 1. Deploying the declared read replica would route the wide aggregate reads (reconciliation reports, monthly revenue, the dispatch dashboard) away from the write-heavy primary. Finally, a pre-insert matching-event table (request, driver offer, dispatch outcome) would close the D-05 scope gap and bring dispatch analytics within reach of the same five-entity core; re-modelling that event stream against a document store (NoSQL comparison) would be the natural next experiment.

References

[1] CDS 534 Teaching Team. Cds 534 database management group project requirement. Course handout, 16 pp. (group shared drive), 2026. Official DDL, queries, and grading rubric for Topic 5.

[2] New York City Taxi and Limousine Commission. Tlc trip record data. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page, 2026. Accessed 2026-10-22. High Volume FHV monthly records and taxi-zone lookup table, used only as a distributional reference for our synthetic generator (decision D-10); the page carries no data-specific open license, and NYC.gov Terms of Use apply.

[3] Uber Engineering. Why uber engineering switched from postgres to mysql. Uber Engineering Blog, https://eng.uber.com/postgres-to-mysql-migration/, 2016.

Recorded as source 2 of the A1 business-flow memo (2026-10-22); core-transaction store practice cited in §1: every charge must be a durable transactional record.

[4] Uber Technologies. Am i charged for cancelling a trip? (uber help). https://help.uber.com/en/riders/article/am-i-charged-for-cancelling-a-trip-?nodeId=5f6415dc-dfdb-4d64-927a-66bb06bc4f82, 2026. Accessed 2026-10-22. First-party wording on cancellation-fee trigger and purpose; the real grace period and fee vary by city — CNY 8 is a course simplification (decision D-06).

[5] Natalia Zuniga-Garcia, Natalia Ruiz Juri, Randy Machemehl, James Scott, and Mauricio Tec. Evaluation of ride-sourcing search frictions and driver productivity: A spatial denoising approach. RideAustin rides dataset, Mendeley Data, V1, 2019. Approx. 1.49M ride-sourcing trips, 2016-06-02 – 2017-04-13. License CC0 1.0. Distributional calibration source for our synthetic generator (decision D-10): tier mix, per-trip ratings (92.6% non-null), fare rates. Verified 2026-10-22.

[6] Natalia Zuniga-Garcia, Natalia Ruiz Juri, Randy Machemehl, James Scott, and Mauricio Tec. Evaluation of ride-sourcing search frictions and driver productivity: A spatial denoising approach. *Transportation Research Part C: Emerging Technologies*, 2019. Accessed 2026-10-22.

A Contribution Statement

B Full Build Script

Data Generation Entry

The dataset is regenerated by `import_data.py` (repository path `02_sql/`): every random choice uses the fixed seed 534, distribution parameters are the RideAustin-calibrated values of decision D-10, and the script ends with built-in acceptance checks (row counts and key ratios), so any member rebuilds identical data from the same repository scripts (constraint C6). Full rebuild on a clean machine:

Listing 11: Rebuild from scratch (idempotent, re-runnable)

```
mysql -u <user> -p < 02_sql/build.sql    # schema v1.0: base + extension + self-checks
python3 02_sql/import_data.py            # fixed seed=534 + built-in acceptance checks
```

Listing 12: build.sql — full DDL with repeatability self-check (schema v1.0 snapshot)

```
-- ============================================================
-- build.sql -- CDS534 group project, Topic 5: full database build script
-- Project: Ride-Hailing Order and Trip Dispatch database
-- (re-runnable: DROP + CREATE restores a clean, identical schema)
--
-- Usage: MySQL Workbench: open this file and execute all (lightning button),
--        or command line: mysql -u <user> -p < build.sql
--
-- Base rule: Section 1 is the official Topic 5 DDL (course handout Section 5,
-- Listing 17; only PDF layout whitespace cleaned, semantics identical). It is
-- never modified; extension tables are appended in Section 3 only.
--
```

Member (SID)	Role	Main Contributions	Load
CHEN Yinan (3162550)	A	Business modelling: process flow, stakeholder needs, quantified goals G1–G5 and constraints; dataset selection and RideAustin calibration profile; billing and cancellation rules (D-05/D-06); workload profile and consistency trade-offs; author of Section 1	15%
ZHANG Yijie (3162641)	B	ER model v1.0.1 and vector diagram; data dictionary (8 tables, 26 columns); extension proposal D-11; normalization review; ER–DDL reconciliation and signed handoff; author of Section 2 with course-notation figure; Q1/Q2 reviewer	20%
CHEN Yanwu (3162744)	C	`build.sql` (official base + extension zone + self-checks); `import_data.py` with fixed seed and RideAustin calibration (60,000 trips); clean re-run evidence on a second machine (C8); schema v1.0; engineering disciplines D-07/D-08; author of Section 3 and Appendix B	20%
LING Tong (3162677)	D	Query catalogue q01–q10 with interpretations; eight concept answer cards; three experiments (index, JOIN vs subquery, concurrency) with archived screenshots; holdout comparison table; author of Sections 4–5	25%
HU Zhongyuan (3162304)	E	Project management: timeline, milestones and follow-ups; decision registry D-05–D-12; meeting minutes (kickoff, M1, M3); four quality gates (ER / SQL / numbers / format); cover, contribution table and Section 6 co-author; integration and final compilation	20%

Table 11: Team contributions (percentages pending group confirmation)

```
-- Version: schema v1.0 (finalised 2026-11-05; ER v1.0.1 frozen and the
-- ER-DDL reconciliation closed). Structure is unchanged from v1.0-rc; only
-- comments and the version tag were updated.
--
-- Version policy: no forced upgrades; compatibility baseline = the lowest
-- registered member version (8.0.45, see env_record.md). Collation
-- utf8mb4_general_ci works on all versions. Below 8.0, CHECK constraints
-- would be silently ignored (still runs); queries avoid window functions
-- and CTEs in favour of JOIN / subquery forms.
-- Maintainer: C (implementation lead); changes must be committed to GitHub.
--
-- NOTE (display copy): this file is the appendix rendering of 02_sql/build.sql
-- with comments translated to English (report language). DDL and SQL
-- statements are byte-identical to 02_sql/build.sql (schema v1.0).
-- ============================================================

-- 0. Dedicated project database (isolated from personal schemas; group-wide
    name)
DROP DATABASE IF EXISTS ride_hailing;
```

```sql
CREATE DATABASE ride_hailing
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_general_ci;
USE ride_hailing;

-- ============================================================
-- 1. Official base DDL (Listing 17, kept verbatim -- do not modify)
--    Five entities: Passenger / Vehicle / Driver / Trip / Payment
--    Key constraints: Driver.vehicle_id NOT NULL UNIQUE -- each driver is
--    bound to exactly one vehicle, and UNIQUE additionally caps each vehicle
--    at most one driver (the 1:1 on the ER diagram is maximum pairing; the
--    1:N arrow in the course figure is the business direction, unreachable
--    in practice). Payment.trip_id NOT NULL UNIQUE -- each trip has at most
--    one payment, and unpaid trips may exist (exactly what the official Q2
--    anti-join relies on).
-- ============================================================

CREATE TABLE Passenger (
    psg_id INT PRIMARY KEY,
    name   VARCHAR(40) NOT NULL
);

CREATE TABLE Vehicle (
    vehicle_id INT PRIMARY KEY,
    plate      VARCHAR(20) NOT NULL UNIQUE
);

CREATE TABLE Driver (
    driver_id  INT PRIMARY KEY,
    name       VARCHAR(40) NOT NULL,
    vehicle_id INT NOT NULL UNIQUE,
    FOREIGN KEY (vehicle_id) REFERENCES Vehicle (vehicle_id)
);

CREATE TABLE Trip (
    trip_id    INT PRIMARY KEY,
    psg_id     INT NOT NULL,
    driver_id  INT NOT NULL,
    fare       DECIMAL(8,2),
    status     VARCHAR(20),
    start_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (psg_id)    REFERENCES Passenger (psg_id),
    FOREIGN KEY (driver_id) REFERENCES Driver (driver_id)
);

CREATE TABLE Payment (
    pay_id  INT PRIMARY KEY,
    trip_id INT NOT NULL UNIQUE,
    amount  DECIMAL(8,2) CHECK (amount >= 0),
    method  VARCHAR(20),
    FOREIGN KEY (trip_id) REFERENCES Trip (trip_id)
);

-- ============================================================
-- 2. Official three queries (Listing 18-20, base verification)
--    On a fresh empty database, empty result sets already count as
--    "runs through" (syntax, constraints and joins are valid).
-- ============================================================

-- Q1 completed trips and their payments (1:1 join)
SELECT t.trip_id, p.name AS passenger, t.fare, py.amount, py.method
FROM Trip t
JOIN Passenger p ON p.psg_id = t.psg_id
```

```
JOIN Payment py  ON py.trip_id = t.trip_id
WHERE t.status = 'completed';

-- Q2 drivers of completed trips not yet paid (NOT EXISTS anti-join)
SELECT d.name
FROM Driver d
JOIN Trip t ON t.driver_id = d.driver_id
WHERE t.status = 'completed'
  AND NOT EXISTS (SELECT 1 FROM Payment py WHERE py.trip_id = t.trip_id);

-- Q3 cumulative income per vehicle
SELECT v.plate, SUM(t.fare) AS earned
FROM Vehicle v
JOIN Driver d ON d.vehicle_id = v.vehicle_id
JOIN Trip t   ON t.driver_id = d.driver_id
GROUP BY v.vehicle_id, v.plate;

-- ============================================================
-- 3. Extension zone (C3, schema v1.0; built 2026-10-25, finalised 11-05)
--    Shape per D-11 (both extensions adopted) + B4 data dictionary:
--    extension attributes live in separate tables keyed by trip_id; the
--    official five tables gain no columns (base rule).
--
--    ER cardinality <-> SQL constraint mapping:
--      ServiceTier 1-N Trip      <-> trip_detail.service_tier_id NOT NULL + FK
--      PickupZone   1-N Trip      <-> trip_detail.zone_id NOT NULL + FK
--      Trip         1-1 TripDetail <-> trip_detail.trip_id is both PK and FK
--                                      (PK-is-FK -> at most one extension row
--                                      per Trip)
--      Cardinalities inside the official five tables are unchanged (1:1 on
--      the diagram = maximum pairing; optionality is what UNIQUE encodes):
--      Vehicle 1:1 Driver via vehicle_id UNIQUE -> each vehicle has 0..1
--      drivers; Trip 1:1 Payment via trip_id UNIQUE -> each trip has 0..1
--      payments (the Q2 anti-join needs unpaid rows to exist).
--
--    Generator-side invariants (import_data.py / C4; not schema-enforced):
--      (1) every Trip has exactly one trip_detail row, cancelled rows
--          included: tier and zone are request attributes fixed at dispatch
--          (q09 counts all Trip rows per D-11 item 1);
--      (2) rating is non-NULL only for status='completed' rows (~90%
--          coverage, D-11 item 2; AVG ignores NULLs automatically). This
--          rule is cross-table semantics: a CHECK cannot reference another
--          table's column and triggers are beyond course scope, so the
--          generator guarantees it and the B4 data dictionary records it
--          (discussed in the report as a boundary of declarative constraint
--          expressiveness).
--
--    Dimension seeds (4 + 15 rows) ship with build.sql rather than the
--    import script: tier and zone names are course settings (D-11), part of
--    schema semantics rather than workload data; identical across machines
--    without Python, and visible in C5 screenshots on an empty database.
-- ============================================================

-- 3.1 Service tier dictionary (names keep RideAustin's native English terms;
--     do not translate to local ride-hailing product names)
--     Calibrated mix: REGULAR ~95.0% / SUV ~3.7% / PREMIUM ~0.9% /
--     LUXURY ~0.3% (RideAustin full-data measurement). fare_multiplier is a
--     descriptive column = measured relative multiple of mean total_fare;
--     generated fares use the four real rate cards directly (A4 section 3),
--     not this column.
CREATE TABLE service_tier (
    tier_id          TINYINT UNSIGNED PRIMARY KEY,
    name             VARCHAR(16)  NOT NULL UNIQUE,
```

```
    fare_multiplier  DECIMAL(3,1) NOT NULL
);

INSERT INTO service_tier (tier_id, name, fare_multiplier) VALUES
    (1, 'REGULAR', 1.0),
    (2, 'SUV',     1.9),
    (3, 'PREMIUM', 2.1),
    (4, 'LUXURY',  2.7);

-- 3.2 Pickup zone dictionary (course setting: ~15 zones, top-3 ~35% share;
--     spatial concentration referenced from NYC TLC; RideAustin has no zone
--     field, hence no real calibration source -- the report declares the
--     zone distribution as a course setting per D-11 item 1. Names are
--     placeholders: rename freely, keep zone_id)
CREATE TABLE pickup_zone (
    zone_id  TINYINT UNSIGNED PRIMARY KEY,
    name     VARCHAR(32) NOT NULL UNIQUE
);

INSERT INTO pickup_zone (zone_id, name) VALUES
    (1,  'Downtown'),
    (2,  'East Austin'),
    (3,  'South Congress'),
    (4,  'South Lamar'),
    (5,  'West Campus'),
    (6,  'Riverside'),
    (7,  'Hyde Park'),
    (8,  'Mueller'),
    (9,  'Airport'),
    (10, 'North Loop'),
    (11, 'Crestview'),
    (12, 'Cherrywood'),
    (13, 'Barton Hills'),
    (14, 'Zilker'),
    (15, 'Tarrytown');

-- 3.3 Trip extension table (1:1: PK-is-FK, at most one row per Trip; carries
--     both D-11 extensions -- service tier x rating and pickup zone -- so
--     q09/q10 each need a single JOIN)
CREATE TABLE trip_detail (
    trip_id         INT PRIMARY KEY,              -- PK-is-FK -> 1:1 with Trip
    service_tier_id TINYINT UNSIGNED NOT NULL,    -- request attribute, fixed at
        dispatch
    zone_id         TINYINT UNSIGNED NOT NULL,    -- pickup zone (q09 hot-zone
       stats)
    rating          TINYINT UNSIGNED NULL,        -- passenger rates driver, 1-5
        stars;
                                                   -- non-NULL only for
                                                      completed rows (~90%)
    FOREIGN KEY (trip_id)         REFERENCES Trip (trip_id),
    FOREIGN KEY (service_tier_id) REFERENCES service_tier (tier_id),
    FOREIGN KEY (zone_id)         REFERENCES pickup_zone (zone_id),
    CONSTRAINT chk_rating CHECK (rating BETWEEN 1 AND 5)
);

-- ============================================================
-- 4. Self-check (for screenshots): version + table list + row counts
-- ============================================================
SELECT VERSION() AS mysql_version;
SHOW TABLES;
-- Row counts right after this script (before data import): base five
-- tables = 0; dimension tables already seeded = service_tier 4 /
-- pickup_zone 15; trip_detail is generated together with Trip by C4 = 0.
```

```
SELECT 'Passenger'   AS tbl, COUNT(*) AS rows_ FROM Passenger
UNION ALL SELECT 'Vehicle',      COUNT(*) FROM Vehicle
UNION ALL SELECT 'Driver',       COUNT(*) FROM Driver
UNION ALL SELECT 'Trip',         COUNT(*) FROM Trip
UNION ALL SELECT 'Payment',      COUNT(*) FROM Payment
UNION ALL SELECT 'service_tier', COUNT(*) FROM service_tier
UNION ALL SELECT 'pickup_zone',  COUNT(*) FROM pickup_zone
UNION ALL SELECT 'trip_detail',  COUNT(*) FROM trip_detail;
```